



WEB APPLICATION ARCHITECTURE – AN OVERVIEW

PROGRAMMING APPLICATIONS AND FRAMEWORKS (IT3030)

LEARNING OUTCOMES

- After completing this lecture, you will be able to,
 - Describe the main components of three tier architecture for web development.
 - Identify some suitable technologies to develop the presentation layer and the application layer of a web application.
 - Apply suitable architectural components to design the presentation layer and the application layer of a Three tier web application.

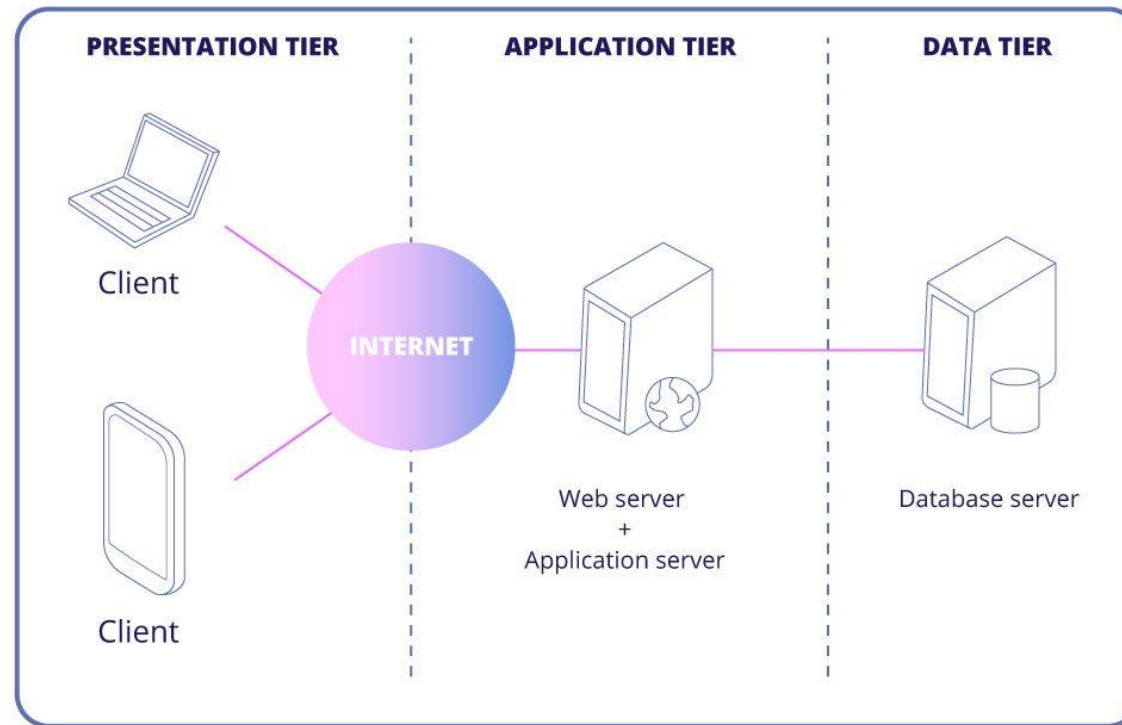
CONTENTS

- Website vs Web Application
- Three Tier/ Three Layer Architecture
- Presentation Layer
 - Some technologies
 - Architecting the Presentation Layer
- Application Layer
 - Some technologies
 - Application Programming Interface (API)
 - Architecting the Application Layer
 - Big Ball of Mud: an architectural anti-pattern
- Other Helpful Technologies

WEBSITE VS. WEB APPLICATION

- What is the difference between a Website and a Web Application?
 - A Website provides static content which can be consumed by the end users.
 - A **Web Application** is designed for **interaction** with the end users.
- Our focus is on **Web Applications**.

THREE TIER ARCHITECTURE



Source: <https://mobidev.biz/blog/web-application-architecture-types>

THREETIER ARCHITECTURE

- Modern web applications are engineered based on this architecture.
- Has three tiers.
 - Presentation tier/ layer
 - Application tier/ layer
 - Database tier/ layer
- The most popular form of n-tier (multitier) architecture.

THREE TIER ARCHITECTURE

- Three tier architecture is a good choice for the development of web applications.
- Reasons?
 - Each tier/ layer runs on its own infrastructure.
 - Each tier/ layer can be developed independently/ in parallel by different teams.
 - Allows updating and scaling up/ down each tier independently without impacting the other tiers.
 - Better security when compared with two tier architecture (due to the isolation of data and logic from the presentation tier).

PRESENTATION LAYER

- Also called the client side or the frontend.
- Technologies used to implement include,
 - HTML/ CSS/ JavaScript
 - React.js
 - Angular
 - Vue.js
 - Some legacy technologies
 - jQuery and AJAX

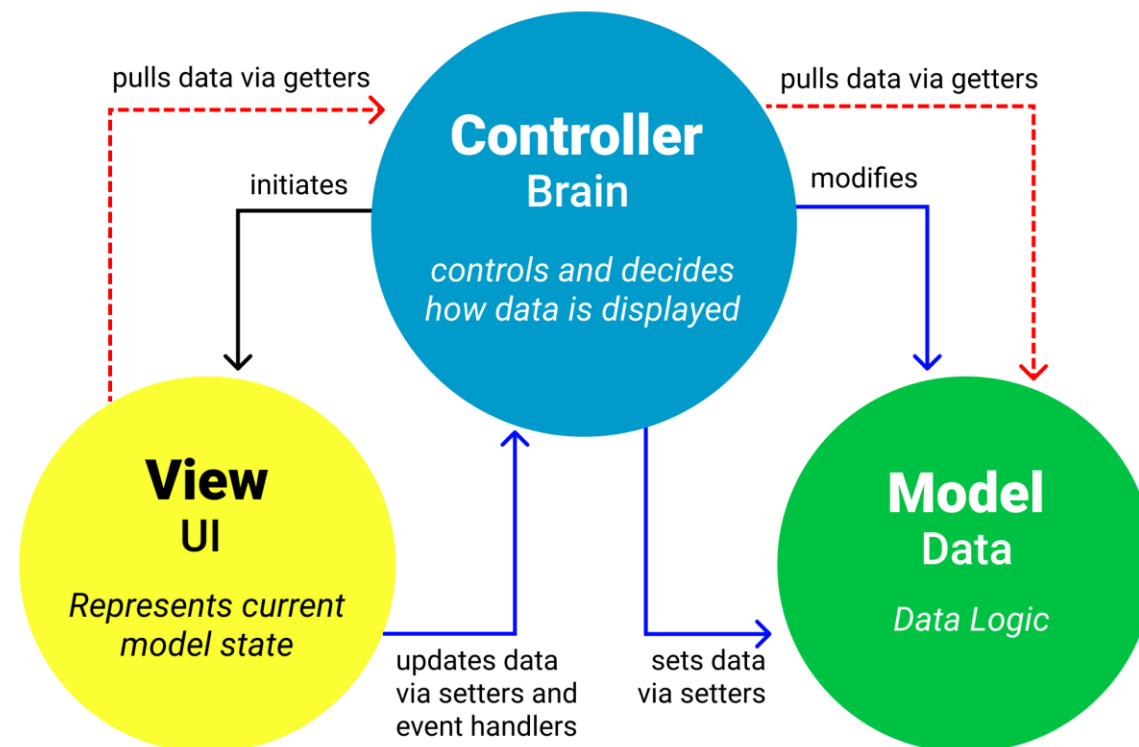
ARCHITECTING THE PRESENTATION LAYER

- Some well known examples
 - Model-view-controller (MVC)
 - Model-view-viewmodel (MVVM)
- Some more examples
 - Model-view-presenter (MVP)
 - Component Architecture
 - Micro frontends

ARCHITECTING THE PRESENTATION LAYER: MVC

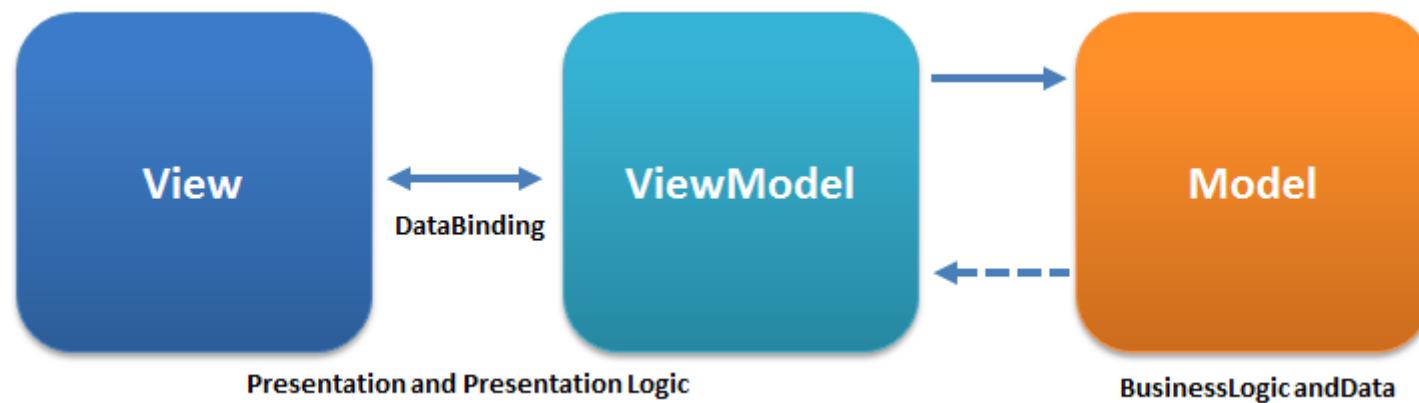
- [Youtube] MVC Explained

MVC Architecture Pattern



Source: <https://www.freecodecamp.org/news/the-model-view-controller-pattern-mvc-architecture-and-frameworks-explained/>

ARCHITECTING THE PRESENTATION LAYER: MVVM



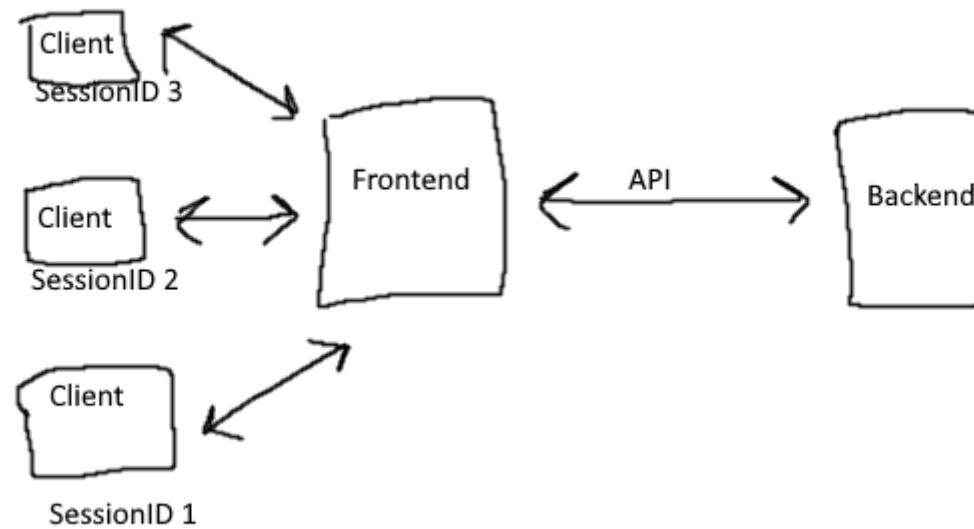
Source: <https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93viewmodel>

APPLICATION LAYER

- Also called the server side or the backend.
- Technologies used to implement include but not limited to,
 - Programming languages and frameworks
 - Java (frameworks: Spring/ Spring Boot)
 - Node.js (frameworks: express.js/ Koa)
 - Python (frameworks: Django/ Flask)
 - PHP (frameworks: Laravel, Symphony)
 - .Net Framework
 - Virtual Machines (VMs)
 - Serverless Computing
 - Containers

APPLICATION LAYER

- How does the Backend and the frontend communicate?



Source: <https://stackoverflow.com/q/71033687>

APPLICATION LAYER: APIS

- Application Programming Interface (API)!
- What is it?
 - A software interface which facilitates communication between two or more applications.
 - Abstracts away the complexities of application integrations.
 - A contract of services offered.

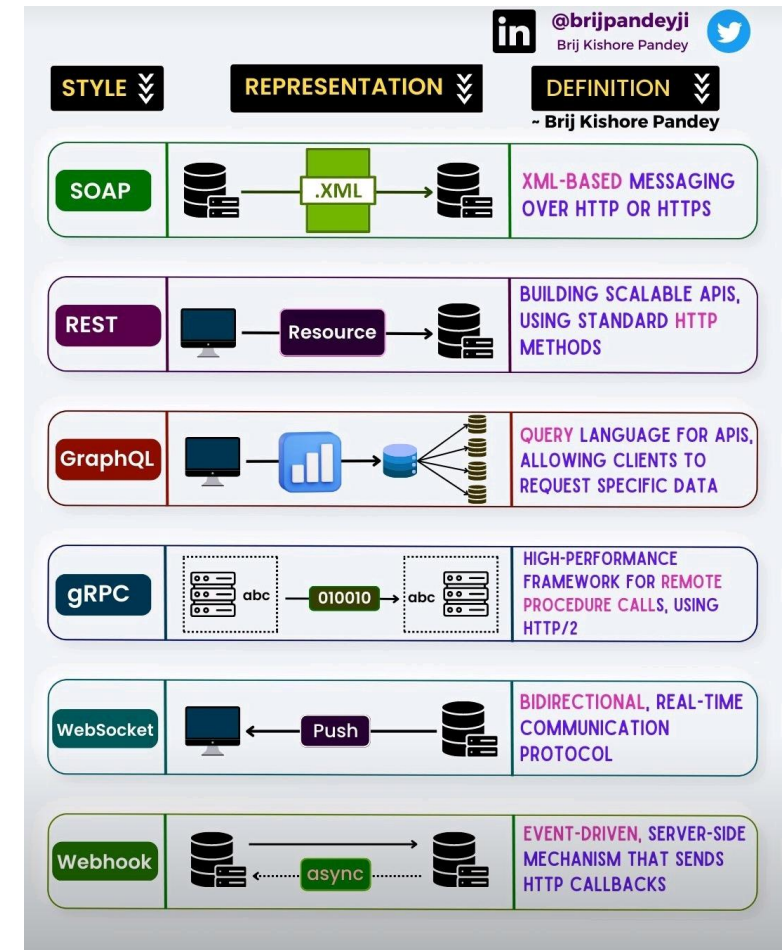
APPLICATION LAYER: APIS

- Benefits of APIs?
 - developers don't have to create everything from scratch - can use existing functions exposed as an API resulting in more productivity.
 - APIs significantly reduce development costs by reducing development efforts.
 - Improves collaboration and connectivity across the ecosystem.

APPLICATION LAYER: APIS

- Different types of APIs?
 - Representational State Transfer API (REST API)
 - Simple Object Access Protocol API (SOAP API)
 - Remote Procedure Calls (RPC)
 - WebSockets
 - GraphQL APIs

More on some of these in a separate lecture.

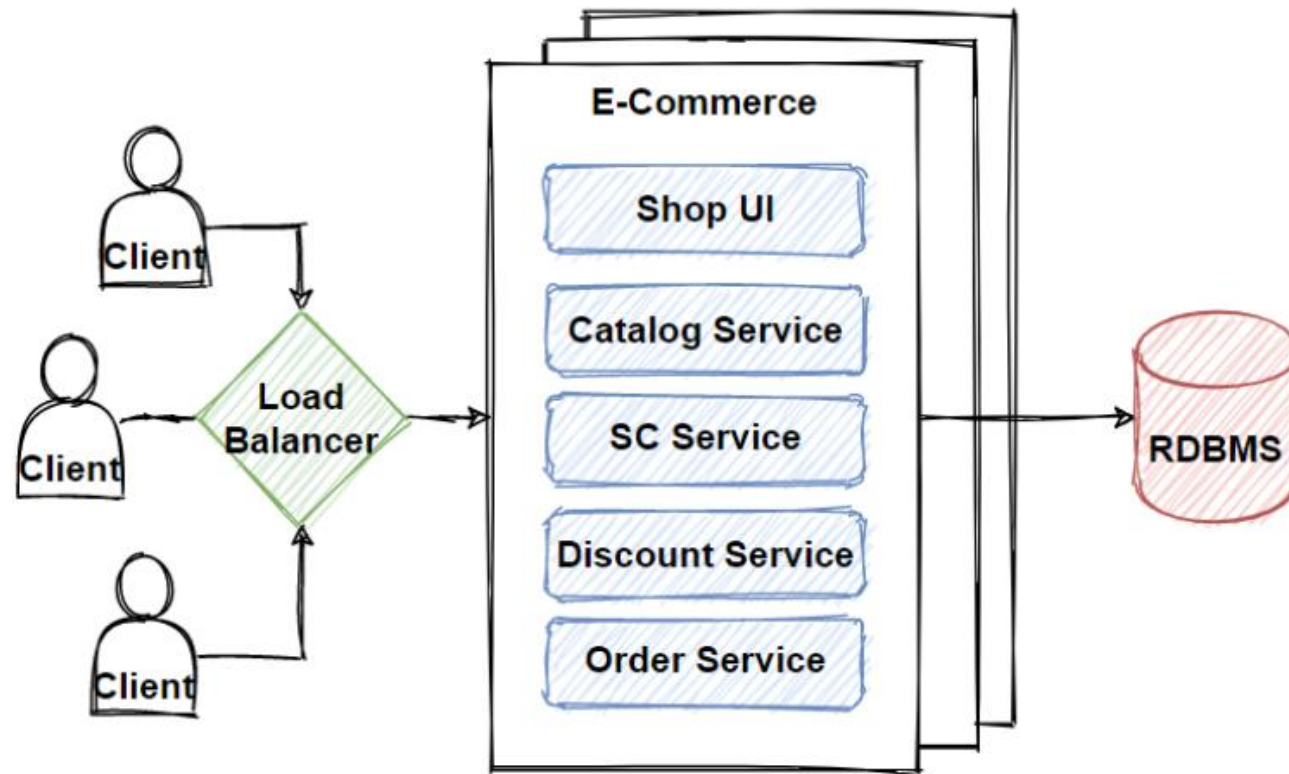


ARCHITECTING THE APPLICATION LAYER

- Some well known examples
 - Monolithic Architecture
 - Microservices Architecture
 - Service Oriented Architecture (SOA)
 - [\[Read\] What is SOA?](#)

ARCHITECTING THE APPLICATION LAYER: MONOLITHIC

Monolithic Architecture



Source: <https://medium.com/design-microservices-architecture-with-patterns/monolithic-architecture-is-still-worth-at-2021-98bfc112dc24>

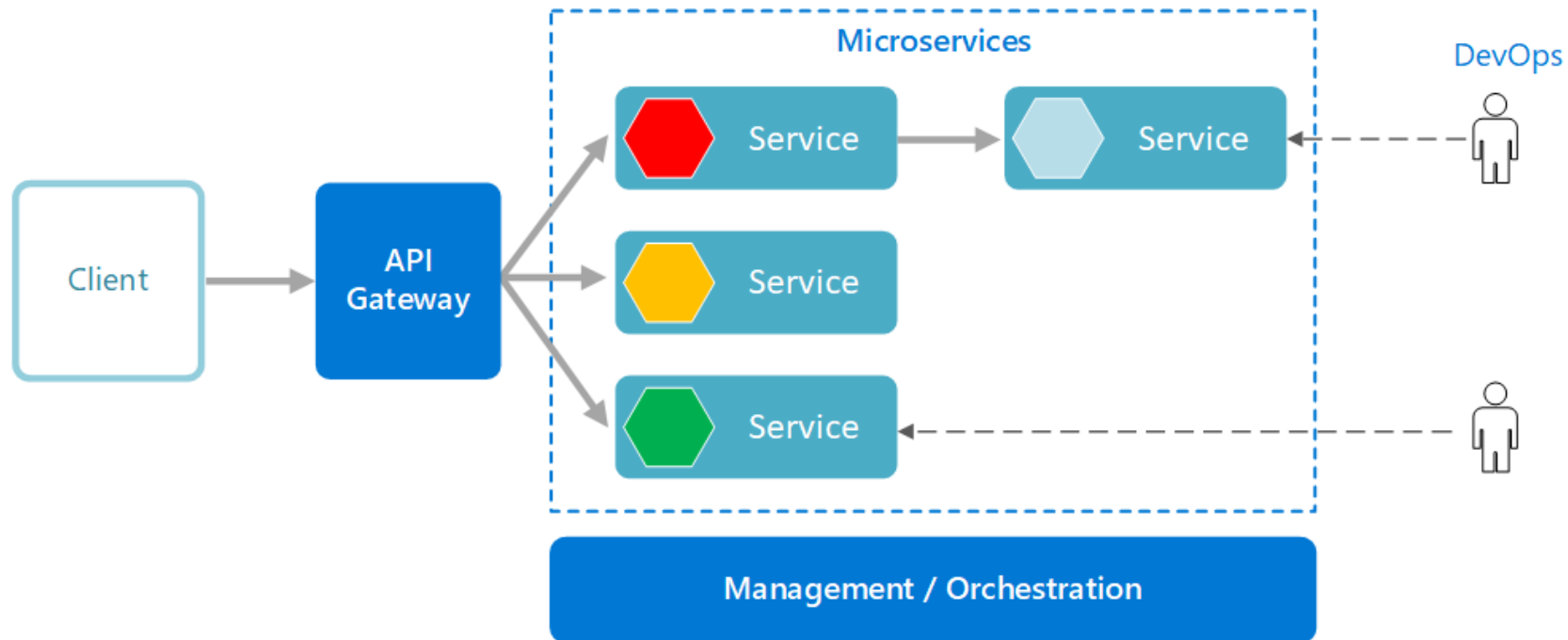
ARCHITECTING THE APPLICATION LAYER: MONOLITHIC

- Drawbacks in Monolithic Architecture?
 - Slower development speed due to **local complexity**.
 - Can't scale individual components.
 - Reliability – if there's an error in any module, it could affect the entire application's availability.
 - Barrier to technology adoption – any changes in the framework or language affects the entire application, making changes often expensive and time-consuming.
 - Lack of flexibility – a monolith is constrained by the technologies already used in the monolith.
 - A small change to a monolithic application requires the redeployment of the entire monolith.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- “Microservices architecture is an approach in which a single application is composed of many loosely coupled and independently deployable smaller services.” (Source – IBM)

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES



Source: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices>

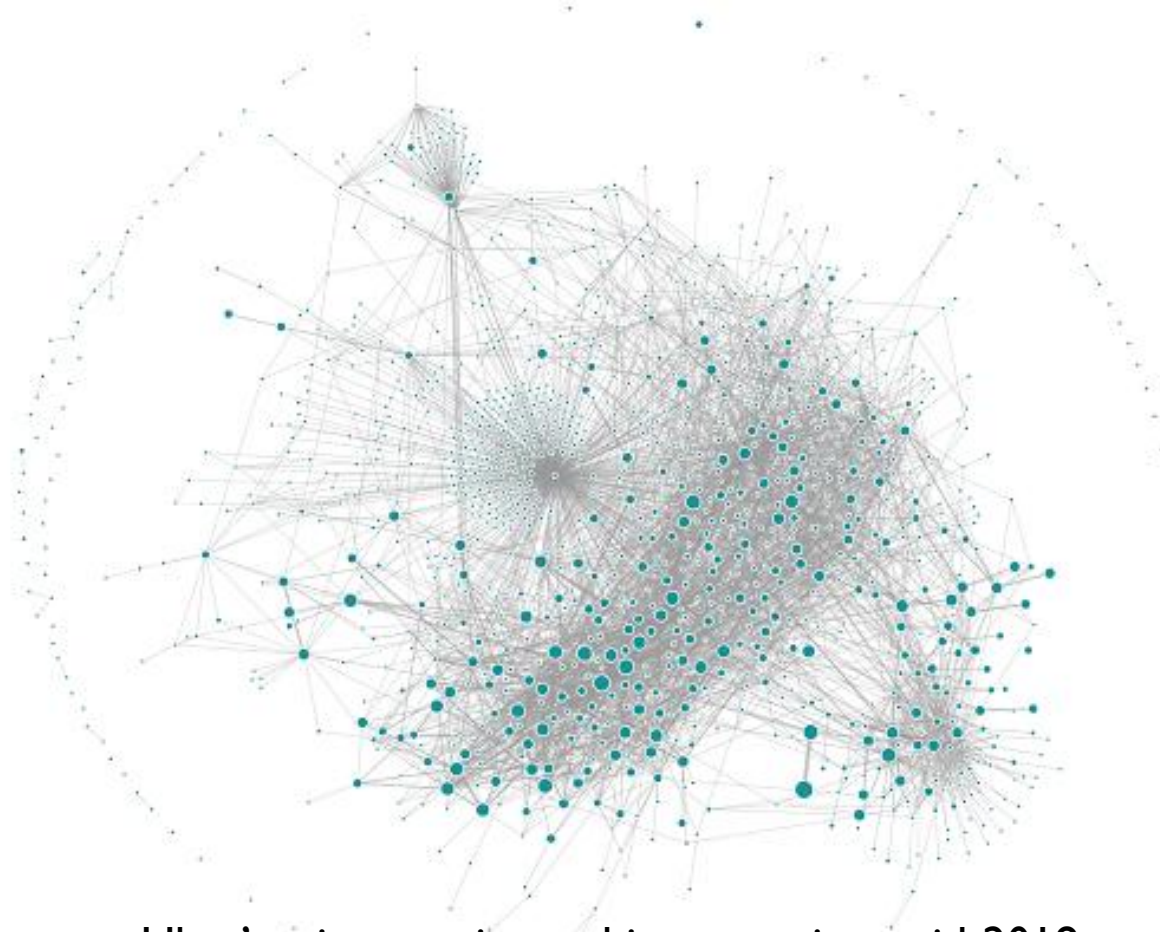
ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- Some benefits of Microservices
 - Microservices are **small, independent, and loosely coupled** (therefore, a single team can write/ maintain a single service).
 - Services can be **deployed independently** (can update an existing service without rebuilding/ redeploying the entire application).
 - Can **scale** each service independently.
 - High **reliability**.
 - Supports **polyglot programming**.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

- Is everything great with Microservices?

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES



Uber's microservice architecture circa mid-2018

Source: <https://www.uber.com/en-LK/blog/microservice-architecture/>

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

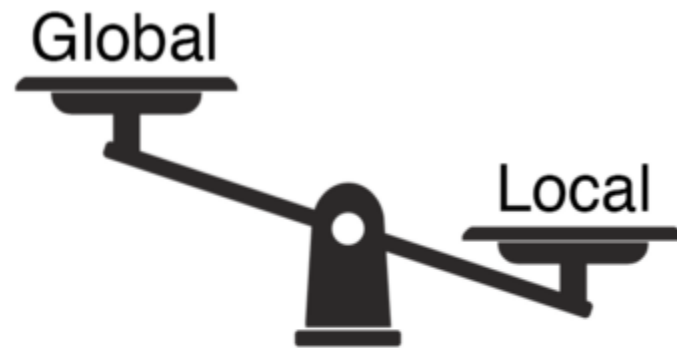
- **High global complexity** is the biggest problem when it comes to Microservices.
 - *Local complexity* depends on the **implementation of a service**; *global complexity* is defined by the **interactions and dependencies** between the services.
- Managing complexity is a constant challenge in Microservices.

ARCHITECTING THE APPLICATION LAYER: MICROSERVICES

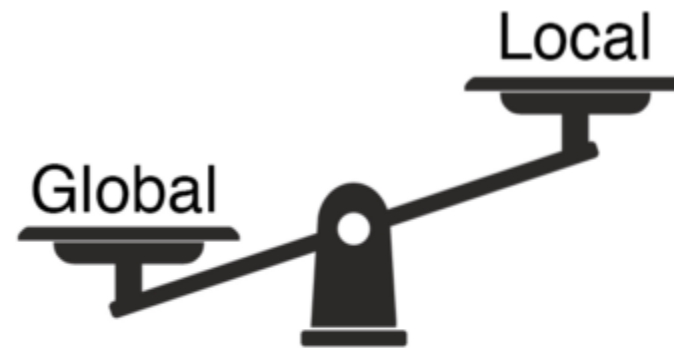
- What are some other downsides of Microservices?
 - High infrastructure costs
 - Debugging challenges
 - A lot more...



BIG BALL OF MUD: AN ARCHITECTURAL ANTI-PATTERN



Big Ball of Mud



Distributed
Big Ball of Mud

Source: <https://www.doit.com/untangling-microservices-or-balancing-complexity-in-distributed-systems/>

OTHER HELPFUL TECHNOLOGIES

- API Gateways (authentication, throttling, orchestration, caching, statistics etc.)
- Content Delivery Networks (CDN)
- Caching Tools (e.g., Redis)
- Load Balancers
- Message Queues
- Cloud Computing
 - Cloud Storage
 - Virtual Machines (VMs)
 - Many more resources...

SOME INTERESTING TOPICS

- Component Based Architecture
- Micro Frontends – Microservices for the frontend
- BFF: Backend For Frontend also known as BIF: Backend In Frontend

REFERENCES

1. [Web application vs. website: finally answered](#)
2. [What is three-tier architecture?](#)
3. [SOAP APIs vs REST APIs](#)
4. [Micro Frontends: What are They and When to Use Them?](#)
5. [Model-view-presenter](#)
6. [An In-Depth Guide to Component-Based Architecture: Features, Benefits, and more](#)
7. [The Complete Guide To BFF \(Backend For Frontend\)](#)
8. [Microservices vs. monolithic architecture](#)
9. [Untangling Microservices or Balancing Complexity in Distributed Systems](#)
10. [Big Ball of Mud - The Daily Software Anti-Pattern](#)



THANK YOU

VISHAN.J@SLIIT.LK

NELUM.A@SLIIT.LK